

Foundation models for medicine & AI for science

A six-page study plan | 30 September 2026

Build practical fluency in model training, inference and post-training, then apply it through a small biomedical reproduction. Use visual intuition, code, controlled experiments and an open-source contribution as successive learning steps.

Suggested effort: 15-20 hours/week for an eight-week first pass, or 7-10 hours/week over 16 weeks. This covers selected lessons and small experiments, not every course in full. Add 2-4 preparation weeks if Python, tensors or basic calculus need work.

A / Visual learner and fundamentals first - recommended

[3Blue1Brown neural networks](#) > [micrograd](#) > [attention](#) > [Build GPT](#) > [GPT-2 reproduction](#) > [CS336 lectures + assignments 1-2](#) > [CMU inference](#) > [Lambert post-training](#) > [OLMo-core](#) or [Marin](#) > [biomedical contribution](#)

Follow weeks 1-8 in order. Insert the makemore lessons when tensor operations, loss functions or gradients remain unclear.

B / Systems and inference first

[Build GPT](#) > [CS336 resource accounting, kernels and parallelism](#) > [CMU inference](#) > [CS336 inference video](#) > small serving benchmark > [CS336 data/scaling](#) > [OLMo-core](#) / [Marin](#) > [Lambert](#) > biomedical project

Best when you already train small models. Prioritize weeks 2-5, then 6-8; return to week 1 if backpropagation is a gap. Benchmark time to first token, output tokens/second and memory under matched conditions.

C / Biology and medicine apprenticeship first

[Attention](#) + [Build GPT](#) > one small example in [MarinFold](#), [Helico](#), [MarinDNA](#) or [OLMo-core](#) > reproduce an experiment > study [CS336](#), [CMU](#) and [Lambert](#) as needed > contribute evidence > attempt an oncology extension.

Best when a specific repository or biology question motivates you. Reproduce a bounded result before proposing a larger research direction.

Before starting

Be able to write/debug Python, multiply matrices, apply the chain rule, explain log probabilities and separate training from evaluation. Learn enough PyTorch to inspect shapes, backpropagate a scalar loss and update parameters.

Your weekly habit

Study 4-6 h; implement/experiment 7-10 h; evaluate/write 4 h. For every 2-3 h of video, spend at least 2 h with code. Keep a learning log: concept, code path, prediction, observation, failure and next experiment.

Resource links are clickable. PDF viewers decide whether they open in new tabs. The companion web page explicitly requests new tabs for resource links.

Study links: [Full web guide](#) | [Source Google Doc](#)

Weeks 1-4 / Build the foundations

1 / Forward pass, loss and backpropagation

Study links: [3Blue1Brown neural networks](#) | [Karpathy micrograd](#)

Watch the visual introductions, then trace a scalar computation forward and backward by hand. Reconstruct a tiny autograd engine, check gradients against finite differences and train a small network. Compare two learning rates with the same initialization.

Deliver: A forward/loss/backward/update diagram and a gradient-check notebook.

Checkpoint: Explain the chain rule, accumulation of gradients, zeroing gradients and when the optimizer changes parameters.

2 / Attention and a tiny transformer

Study links: [3Blue1Brown attention](#) | [Build GPT](#) | [makemore lessons](#)

Implement single-head causal attention, then multiple heads and an MLP block. Print B, T, C; Q/K/V; scores; attention weights; and logits. Check that changing future tokens leaves earlier logits unchanged in evaluation mode. Train a tiny character model and compare sampling temperatures.

Deliver: Annotated tensor-shape diagram, causal attention code and generation examples.

Checkpoint: Explain masking, attention scaling, residual connections and logits versus probabilities. Use makemore for gaps in tensor or loss intuition.

3 / Real pretraining engineering

Study links: [GPT-2 reproduction](#) | [CS336 course/assignments](#) | [CS336 recordings](#)

Use the video as an engineering reference; a full 124M-parameter reproduction is not required. Record model/config, tokenizer, dataset split, optimizer and token budget. Run a tiny baseline, then change one factor at a time: learning rate, batch size, context or width. Save and reload a checkpoint.

Deliver: Baseline plus two comparisons, loss curves, tokens/second, peak memory and a resume note.

Checkpoint: Explain effective batch size, accumulation, held-out validation and which comparisons change more than one variable.

4 / GPU efficiency and distributed training

Study links: [CS336 GPUs, kernels and parallelism](#) | [Lecture recordings](#)

Account for parameters, gradients, optimizer states, activations and temporary memory. Study BF16/FP16/TF32, gradient accumulation, FlashAttention, and data/tensor/pipeline parallelism. Profile attention and the MLP; compare reference attention with supported optimized attention after checking correctness.

Deliver: A memory/FLOP sheet and benchmark recording hardware, shapes, dtype, warm-up and timing method.

Checkpoint: Explain why checkpointing, reduced precision and accumulation address different bottlenecks. On unsupported hardware, finish accounting and defer GPU/Triton benchmarks explicitly.

Weeks 5-8 / Engineer, adapt, contribute

5 / Inference and KV caches

Study links: [Stanford inference video](#) | [CMU inference course](#)

Implement uncached and KV-cached autoregressive generation. Under deterministic settings, compare outputs and logits with a stated numerical tolerance. Vary prompt and output lengths; measure prefill/decode latency, time to first token, output tokens/second and cache memory. Study MHA/MQA/GQA, batching, PagedAttention, prefix caching, quantization and speculative decoding.

Deliver: Request-to-prefill-to-cache-to-decode diagram plus correctness and latency/memory results.

Checkpoint: Distinguish latency from throughput and explain why a cache saves work while consuming memory.

6 / Data, scaling and biomedical adaptation

Study links: [CS336 data/scaling](#) | [Ai2 training docs](#)

Create a data manifest with provenance, license and document-level splits. Inspect duplicates and contamination. Design a general:biomedical mixture sweep of 100:0, 95:5, 80:20 and 50:50 with equal total tokens and matched settings; execute only what fits your budget. Study small scaling-law fits without assuming extrapolation will hold.

Deliver: A written experiment design, data-quality report, baseline and optional small mixture results.

Checkpoint: Measure biomedical gains and general regressions. Explain continued pretraining versus SFT.

7 / SFT, preferences and reasoning RL

Study links: [Lambert course](#) | [RLHF book](#) | [CS336 assignment 5](#) | [Tulu](#)

Study instruction tuning, reward modeling, rejection sampling, policy gradients, DPO and RLVR. SFT a small model on a narrow, license-compatible dataset. Compare the same held-out prompts before/after. Add a preference method only after the baseline works. Use a truly verifiable toy task for initial RLVR experiments.

Deliver: Before/after table and errors covering facts, omissions, unsupported claims and format adherence.

Checkpoint: Explain policy, reference model, reward, KL and preference pairs. Reward gains do not establish clinical correctness.

8 / Reproduce and enter an open lab

Study links: [Marin](#) | [OLMo-core](#) | [MarinFold](#) | [Helico](#) | [MarinDNA](#) | [AstaBench](#)

Choose one repository. Read setup instructions and one complete experiment/issue discussion, including failures. Pin a commit and reproduce a small example or component. Save commands, environment, data/config and metrics. Write expected versus actual results and one discriminating next experiment.

Deliver: A reproducible experiment note and one scoped contribution proposal.

Checkpoint: Another researcher can follow your record. If the full run is too costly, reproduce preprocessing, a metric or an evaluation subset and state the narrower scope.

Complete course & material list

Use the core stack in sequence; consult the depth courses when an experiment exposes a gap. Links lead to public materials or their official hubs. Availability of lessons does not imply free compute, grading or enrollment.

Visual intuition

Study links: [3Blue1Brown neural networks](#) | [Attention, step by step](#)

Neural networks, gradient descent and backpropagation; then embeddings, Q/K/V, scores, softmax and weighted values. Draw the entire attention path from memory.

From-scratch coding / Andrej Karpathy

Study links: [Zero to Hero syllabus](#) | [micrograd](#) | [Build GPT](#) | [GPT-2 reproduction](#)

makemore: bigram model; MLP; activations/gradients and BatchNorm; manual backpropagation; WaveNet. The syllabus also includes GPT tokenization. Use these for language-model losses, embeddings, shape fluency and training diagnostics.

Foundation-model engineering / Stanford CS336

Study links: [Course, slides, code and assignments](#) | [2026 recordings](#) | [Inference lecture 10](#)

Source roadmap: L1 tokenization; L2 PyTorch/einops, FLOPs, memory and arithmetic intensity; L3 architectures; L4 attention alternatives/MoE; L5 GPUs/TPUs; L6 kernels/Triton; L7-8 parallelism; then scaling, inference, evaluation, data and post-training. Assignments: 1 tokenizer/model/optimizer; 2 profiling, Triton FlashAttention2 and distributed training; 3 scaling; 4 filtering/deduplication; 5 SFT and reasoning RL, with optional DPO/safety alignment.

Inference / CMU and Graham Neubig

Study links: [11-664/763 Inference Algorithms, Fall 2025](#) | [11-711 Advanced NLP 2024](#)

Inference: sampling, beam/A* search, best-of-N, reasoning and test-time compute, prefix sharing, KV caches, speculative decoding, sparse attention, batching and serving. Attempt decoder homeworks. Advanced NLP adds Build Your Own LLaMA, RAG, reimplementation and research-project work as described in the source roadmap.

Post-training / Nathan Lambert

Study links: [Course: videos, slides and PDFs](#) | [Free online book](#)

Source sequence L0-10: ML foundations; overview; instruction tuning/reward models/rejection sampling; RL math; implementation; reasoning/RLVR; DPO; synthetic data/distillation/AI feedback; preference data; over-optimization/reward hacking; regularization/generalization. Optional current additions: L11 tools/agents, L12 evaluation, L13 character training.

Targeted depth / UC Berkeley

Study links: [Understanding LLMs, Spring 2024](#) | [Advanced LLM Agents, Spring 2025](#) | [CS285 Deep RL](#)

Understanding LLMs: scaling, interpretability, reasoning, evaluation, robustness, privacy, unlearning and safety. Agents: search/planning, tool use, code, mathematics and agentic workflows. CS285: selected policy-gradient and actor-critic foundations when RL math blocks progress.

Open laboratories & biomedical directions

Ai2: open training and scientific workflows

Study links: [Developer docs](#) | [OLMo training](#) | [OLMo-core](#) | [Tulu](#) | [AstaBench](#) | [AstaBench code](#)

Trace data/tokenization, config, optimizer, distributed training, checkpoints, evaluation and post-training. OLMo-core is the training-stack entry; Tulu covers post-training; Asta/AstaBench connect to scientific research tasks and evaluation.

Marin / Open Athena

Study links: [Marin repository](#) | [Open Athena](#) | [Projects](#) | [mumwelt](#)

Start with the documented TinyStories/LM-training tutorials where available. Read experiment reports and retrospectives for data mixtures, optimizer decisions, loss spikes, scaling and efficiency. mumwelt is a project-context tool named in the source; inspect its current setup before use.

Structural biology, design and genomics

Study links: [MarinFold](#) | [Helico](#) | [MarinDNA](#) | [BoltzGen and PlantCAD/GeneCAD](#)

MarinFold: protein structure through next-token prediction. Helico: experiments with AF3-like architectures. MarinDNA: genomic language modeling. BoltzGen: target-specific protein generation. PlantCAD/GeneCAD: plant genomics. Learn the native task first; these are not automatically oncology or clinical models.

Choose one small medical or scientific extension

- Biomedical adaptation: compare high-quality domain mixtures with a general baseline; preserve general-domain evaluation.
- Literature agents or trial eligibility: evaluate evidence attribution, structured-field precision/recall, unsupported claims and missing criteria.
- Genomics: reproduce native metrics before testing a cancer-relevant variant task; control split leakage and label provenance.
- Protein/binder design: start with input preparation, a small reproduction or evaluation; computational scores require separate biological validation.
- Later options: tumor single-cell representation plus language, radiotherapy workflow copilots, longitudinal clinical reasoning and virtual-patient evaluation.

Evaluation discipline

Define the task, comparator, held-out split and primary outcome before training. Record factuality, clinical relevance, omission and unsupported-claim rates, terminology/field accuracy, calibration where measurable, subgroups and appropriate clinician review. Keep benchmark questions and close paraphrases out of training. Use public or appropriately licensed data; patient-identifiable data requires a separate governance process.

The source mentions a "CellOLMo-style" cellular/molecular grounding direction. An official named Ai2 project was not verified; retain this as a conceptual interest rather than a confirmed resource.

Pacing, checkpoints & next steps

A slower schedule and a research extension

For 16 weeks, allocate two weeks to each module: the first for lessons and implementation, the second for experiments and the deliverable. For 24 weeks, use three weeks/module at roughly 5-7 hours weekly. Move dates when a checkpoint is unmet rather than doubling the next workload.

- Extension week 9: select one task; record data rights, splits, baseline, outcome and failure criterion.
- Week 10: run the baseline; inspect errors; write an annotation rubric and check agreement where human scoring is needed.
- Week 11: change one factor and record compute, seeds and controls; repeat affordable runs when randomness matters.
- Week 12: use the untouched test set; report denominators, uncertainty where appropriate, regressions, failures and reproducible commands.

Ready-to-contribute checklist

- Explain forward/backward passes and implement causal self-attention with every tensor dimension.
- Explain KV caching, prefill versus decode, latency versus throughput, BF16, accumulation, FlashAttention and basic parallelism.
- Describe filtering, deduplication, contamination, mixture design and the limits of scaling estimates.
- Distinguish pretraining, continued pretraining, SFT, preference optimization, RLHF/RLVR and inference-time scaling.
- Trace an OLMo/Marin config, reproduce a small experiment and propose a testable change.
- Design biomedical evaluation beyond QA accuracy and preserve a clear error taxonomy.

Use a tutor without outsourcing the learning

Predict before running code. Ask for a tensor trace or diagnostic question, then explain the result yourself. Locate the data loader, attention, optimizer, checkpoints and evaluation in a real repository. For coursework, follow its policy: CS336 permits conceptual/low-level assistance but prohibits directly using an LLM to solve assignment problems.

Make an evidence-based first contribution

Choose Marin/OLMo for general model engineering; MarinFold/Helico/MarinDNA for biology; Tulu for post-training; Asta for scientific workflows. Write: "I reproduced X at commit Y with configuration Z. I expected A, observed B and propose C, checked by D." Attach commands and logs before offering broad new ideas.

Sources and verification

Study links: [Google Doc](#) | [Linked conversation](#) | [Full web guide](#)

Compiled from the supplied Google Doc and available conversation entries; effort estimates, checkpoints and project scaffolds are added recommendations. Key official hubs checked on 30 September 2026. CS336 is Spring 2026; the direct inference video is Stanford lecture 10, not CMU. Lambert now lists lectures 11-13. CMU pages yielded little readable text and Berkeley Understanding LLMs could not be retrieved, so those syllabus details remain source-derived. All named source courses and projects are retained; not every video was watched or repository executed.